

高级计数技术

许多计数问题用第6章讨论的方法是不容易求解的。例如：有多少个 n 位比特串不包含两个连续的0？为求解这个问题，令 a_n 是这种 n 位比特串数，给定一个参数，可以证明序列 $\{a_n\}$ 满足 $a_{n+1}=a_n+a_{n-1}$ （其中初始条件为 $a_1=2$ ， $a_2=3$ ）。这个等式叫作递推关系，它和初始条件 $a_1=2$ 、 $a_2=3$ 确定了序列 $\{a_n\}$ 。此外，从这个与序列的项有关的等式可以找到 a_n 的显式公式。正如我们将要看到的，可以用一种类似的技术来求解许多不同的计数问题。

我们将讨论两种在算法研究中最重要递推关系。首先，我们将介绍称为动态规划的重要算法范式。遵循这一个范式的算法将问题分为重叠的子问题。通过递推关系找到子问题的解答方案从而解出原始问题。其次，我们将介绍另一个重要算法范式：分而治之。遵循这一个范式的算法将问题不断分解为固定数量的不重叠的子问题，直到这些子问题被直接解决。这些算法的复杂度可以采用特别的递推关系来分析。在本章中，我们将讨论大量的分而治之算法，并用递推关系来分析它们的复杂度。

我们也将看到，可以用形式幂级数（也叫作生成函数）来求解许多计数问题，其中 x 的幂的系数代表我们感兴趣的序列的项。除了求解计数问题外，生成函数还可用于求解递推关系以及证明组合恒等式。

许多其他类型的计数问题不能使用第6章所讨论的技术求解，例如：有多少种方式把7项工作分给3个雇员而使得每个雇员至少得到一项工作？有多少个素数小于1000？可以通过计数集合并集中的元素数来求解这两个问题。我们将开发一种叫作容斥原理的技术来计数在集合并集中的元素个数，并且将说明怎样用这种技术求解计数问题。

可以用本章学到的技术与第6章的基本技术一起求解许多计数问题。

8.1 递推关系的应用

8.1.1 引言

第2章介绍了怎样递归定义一个序列。一个序列的递归定义指定了一个或多个初始的项以及一个由前项确定后项的规则。这个从某些前项求后项的规则就叫作递推关系。如果一个序列的项满足递推关系，则这个序列就叫作递推关系的解。

本章我们将研究用递推关系解决计数问题。如一群细菌的数目每小时增加一倍。如果开始有5个细菌，在 n 小时末将有多少个细菌？为求解这个问题，令 a_n 是 n 小时末的细菌数。因为细菌数每小时增加一倍，只要 n 是正整数，关系 $a_n=2a_{n-1}$ 就成立。对所有的非负整数 n ，这个关系和初始条件 $a_0=5$ 一起唯一地确定了 a_n 。我们可使用第2章中的迭代方法得到 a_n 的公式，即对于所有非负整数 n ， $a_n=5 \cdot 2^n$ 。

某些计数问题不能用第6章给出的技术求解，但可以通过找到序列的项之间的关系，如在涉及细菌的问题中的关系，即递推关系来求解。我们将研究各种能用递推关系构造模型的计数问题。在第2章中，我们提出解决某些递推关系的方法。我们将在8.2节研究一些方法，针对满足某类递推关系的序列，求出序列的项的显式公式。

本节最后，我们将介绍动态规划的算法范式，在解释这种范式原理之后，将给出例题说明。

8.1.2 用递推关系构造模型

我们可以使用递推关系构造各种问题的模型，例如找复利(见 2.4 节例 11)、计数岛上的兔子、确定汉诺塔难题的移动次数，以及计数具有确定性质的比特串。

例 1 说明了怎样用递推关系建立关于岛上兔子数的模型。

例 1 兔子和斐波那契数 考虑下面的问题，它是由里奥那多·比萨诺，也就是斐波那契，于 13 世纪在《算书》(Liber abaci)一书中提出来的。一对刚出生的兔子(一公一母)被放到岛上。每对兔子出生后两个月才开始繁殖后代。如图 1 所示，在出生两个月以后，每对兔子在每个月都将繁殖一对新的兔子。假定兔子不会死去，找出 n 个月后关于岛上兔子对数的递推关系。

新生的对数 (至少两个月大)	已有的对数 (比两个月小)	月	新生的 对数	已有的 对数	总对数
		1	0	1	1
		2	0	1	1
		3	1	1	2
		4	1	2	3
		5	2	3	5
		6	3	5	8
					

图 1 岛上的兔子

解 用 f_n 表示 n 个月后的兔子对数。我们将证明 $f_n(n=1, 2, 3, \dots)$ 是斐波那契序列的项。

可以用递推关系建立兔子数的模型。在第 1 个月末，岛上的兔子对数是 $f_1=1$ 。由于这对兔子在第 2 个月没有繁殖，所以 $f_2=1$ 。为找到 n 个月后的兔子对数，要把前一个月岛上的对数 f_{n-1} 加上新生的对数——这个数等于 f_{n-2} ，因为每对两个月大的兔子都生出一对新兔子。

因此，序列 $\{f_n\}$ 满足递推关系

$$f_n=f_{n-1}+f_{n-2},\ n\geqslant 3$$

及初始条件 $f_1=1$ 和 $f_2=1$ 。由于这个递推关系和初始条件唯一地确定了这个序列，所以 n 个月后岛上的兔子对数由第 n 个斐波那契数给出。

例 2 涉及一个著名的难题。

例 2 汉诺塔 19 世纪后期由法国数学家埃德沃德·卢卡斯发明的一个流行的游戏叫作汉诺塔，它由安装在一个板上的 3 根柱子和若干大小不同的盘子构成。开始时，这些盘子按照大小的次序放在第一根柱子上，大盘子在底下(如图 2 所示)。游戏的规则是：每一次把 1 个盘子从一根柱子移动到另一根柱子，但是不允许这个盘子放在比它小的盘子上面。游戏的目标是把所有的盘子按照大小次序都放到第二根柱子上，并且将最大的盘子放在底部。

令 H_n 表示解 n 个盘子的汉诺塔问题所需要的移动次数。建立一个关于序列 $\{H_n\}$ 的递推关系。

解 开始时 n 个盘子在柱 1。按照游戏规则，我们可以用 H_{n-1} 次移动将上边的 $n-1$ 个盘子移到柱 3(图 3 说明了此刻的柱子和盘子)。在这些移动中保留最大的盘子不动。然后，我们用一次移动将最大的盘子移到第二根柱子上。我们可以再使用 H_{n-1} 次移动将柱 3 上的 $n-1$ 个

盘子移到柱2, 把它们放到最大的盘子上面, 这个最大的盘子一直放在柱2的底部。这表示解 n 个盘子的汉诺塔问题所需要的移动次数为 $2H_{n-1}+1$ 次。

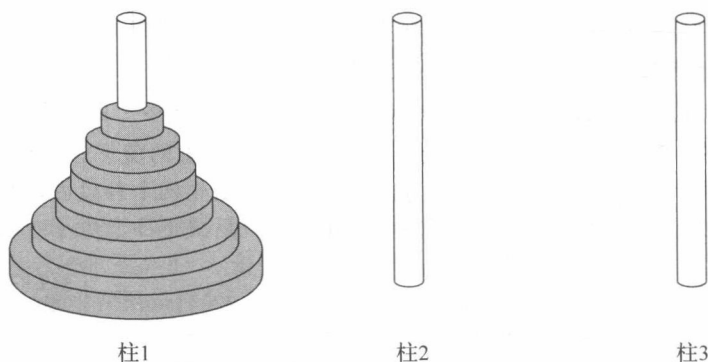


图2 汉诺塔的初始位置

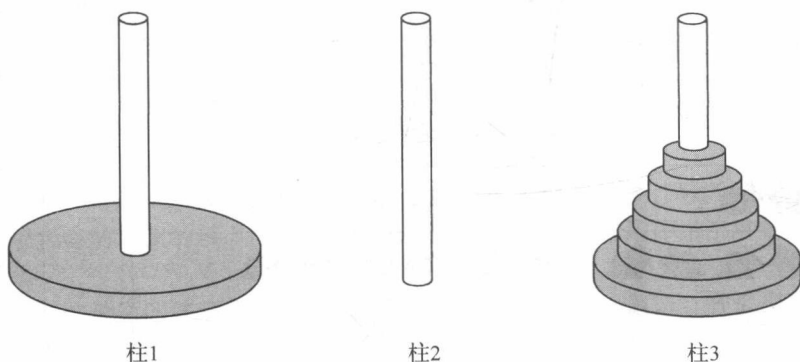


图3 汉诺塔的一个中间位置

我们现在证明解决 n 个盘子的汉诺塔问题的移动次数不能少于 $2H_{n-1}+1$ 次。注意, 在移动最大的盘子之前, 必须要将 $n-1$ 个更小的盘子移动到不是柱1的柱子上, 做这件事需要至少 H_{n-1} 次移动, 另一次必要的移动是将最大的盘子移走。最后, 将这 $n-1$ 个更小的盘子移动到最大的盘子上需要至少 H_{n-1} 次移动。将这些移动次数相加就是完成这个任务的移动次数的下限。

我们的结论是

$$H_n = 2H_{n-1} + 1$$

初始条件是 $H_1=1$, 因为依照规则一个盘子可以用1次移动从柱1移到柱2。

我们可以使用迭代方法求解这个递推关系。

$$\begin{aligned} H_n &= 2H_{n-1} + 1 \\ &= 2(2H_{n-2} + 1) + 1 = 2^2 H_{n-2} + 2 + 1 \\ &= 2^2 (2H_{n-3} + 1) + 2 + 1 = 2^3 H_{n-3} + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1} H_1 + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1 \\ &= 2^{n-1} + 2^{n-2} + \cdots + 2 + 1 \\ &= 2^n - 1 \end{aligned}$$

为了用序列前面的项表示 H_n , 我们重复地用到这个递推关系。在倒数第二个等式中用了初始条件 $H_1=1$ 。最后一个等式是基于等比序列的求和公式得出的, 这个公式可以在2.4节的定理1中找到。

用迭代方法找出了具有初始条件 $H_1=1$ 的递推关系 $H_n=2H_{n-1}+1$ 的解。这个公式可以用

数学归纳法证明。证明留给读者作为本节的练习 1。

关于这个难题有一个古老的传说，在汉诺有一座塔，那里的僧侣按照这个游戏的规则从一个柱子到另一个柱子移动 64 个金盘子，据说当他们结束游戏时世界就到了末日。如果这些僧侣 1 秒移动 1 个盘子，这个世界将在他们开始多久以后终结？

根据这个显式公式，僧侣需要

$$2^{64}-1=18\,446\,744\,073\,709\,551\,615$$

次移动来搬这些盘子。每次移动需要 1 秒，他们将用 5000 亿年来求解这个难题，因此这个世界的寿命应该相当长。

评注 许多人研究了源自例 5 所述汉诺塔难题的各种问题。某些问题用到更多的柱子，某些问题允许同样大小的盘子，某些问题对盘子的移动类型加以限制。一个最古老和最有趣的问题是**雷夫难题**[⊖]，它是 1907 年由亨利·杜德尼在他的《坎特伯雷谜题》(Canterbury Puzzle)一书中提出来的。这个难题是雷夫提出的，他让一个朝圣者把一堆各种大小的奶酪从 4 个凳子中的一个移到另一个，移动中不允许把直径较大的奶酪放在较小的奶酪上面。如果用柱子和盘子的概念来表述雷夫难题，除了使用 4 根柱子之外，其他和汉诺塔的规则一样。类似地，我们可以把汉诺塔问题推广到 $p(p>3)$ 根柱子的情形。(需要说明的是，已有公开声明说解决了这个问题，不过这并未获专家认可。)然而，2014 年 Thierry Bousch 证明了求解 n 个盘子的雷夫难题所需要的最少移动次数，它等于由弗雷姆(Frame)和斯图尔特(Stewart)在 1939 年发明的算法所使用的移动次数。(更详细的信息参见本节末的练习 38~45 和 [St94] 及 [Bo14]。)

Links

例 3 说明了怎样用递推关系计数具有指定长度和某种性质的比特串。

例 3 对于不含 2 个连续 0 的 n 位比特串的个数，找出递推关系和初始条件。有多少个这样的 5 位比特串？

解 设 a_n 表示不含 2 个连续 0 的 n 位比特串的个数。我们将假定 $n \geq 3$ ，比特串至少有 3 位。 n 位比特串可以分为以 1 结尾的和以 0 结尾的。

精确地说，不含 2 个连续 0 并以 1 结尾的 n 位比特串就是在不含 2 个连续 0 的 $n-1$ 位比特串的尾部加上一个 1。因此存在 a_{n-1} 个这样的比特串。

不含 2 个连续 0 并以 0 结尾的 n 位比特串的 $n-1$ 位必须是 1，否则就将以 2 个 0 结尾。因而，精确地说，不含 2 个连续 0 并以 0 结尾的 n 位比特串就是在不含 2 个连续 0 的 $n-2$ 位比特串的尾部加上 10。因此存在 a_{n-2} 个这样的比特串。

如图 4 所示，可以断言对于 $n \geq 3$ ，有

$$a_n = a_{n-1} + a_{n-2}$$

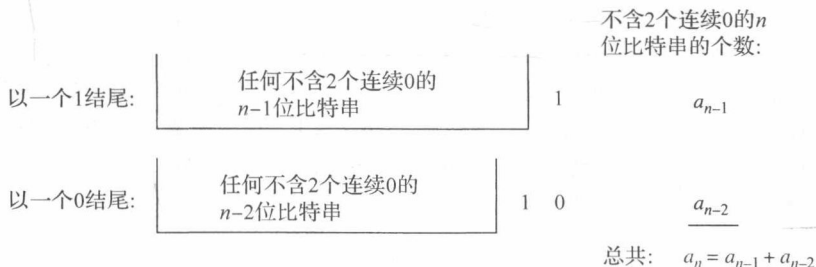


图 4 计数不含 2 个连续 0 的 n 位比特串

初始条件是 $a_1=2$ ，因为 1 位的比特串是 0 或 1，没有连续的 2 个 0。而 $a_2=3$ ，因为 2 位的比特串中满足条件的是 01、10 和 11。使用 3 次递推关系就可得到 a_5 。

⊖ 雷夫(Reve)，更常见的是拼写为 reeve，这个词在古代是指地方长官(governor)。

$$a_3 = a_2 + a_1 = 3 + 2 = 5$$

$$a_4 = a_3 + a_2 = 5 + 3 = 8$$

$$a_5 = a_4 + a_3 = 8 + 5 = 13$$

评注 注意 $\{a_n\}$ 和斐波那契序列满足同样的递推关系。因为 $a_1 = f_3$ 且 $a_2 = f_4$ ，从而有 $a_n = f_{n+2}$ 。

例4说明怎样用递推关系建立编码字数的模型，这种编码字是通过确定的有效性检测所允许的。

例4 编码字的枚举 如果一个十进制数字串包含偶数个0，计算机系统就把它作为一个有效的编码字。例如，1230407869是有效的，而120987045608不是有效的。设 a_n 是有效的 n 位编码字的个数。找出一个关于 a_n 的递推关系。

解 注意 $a_1 = 9$ ，因为存在10个1位十进制数字串，并且只有一个即串0是无效的。通过考虑怎样由 $n-1$ 位的数字串构成一个 n 位有效数字串，就可以推导出关于这个序列的递推关系。从少1位数字的串构成 n 位有效数字串有两种方式。

第一种，在一个 $n-1$ 位的有效数字串后面加上一个非0的数字就可以得到一个 n 位的有效数字串。加这个数字的方式有9种。因此用这种方法构成 n 位有效数字串的方式有 $9a_{n-1}$ 种。

第二种，在一个无效的 $n-1$ 位数字串后面加上一个0就可以得到 n 位有效的数字串。（这将产生具有偶数个0的串，因为无效的 $n-1$ 位数字串有奇数个0。）这样做的方式数等于无效的 $n-1$ 位数字串的个数。因为存在 10^{n-1} 个 $n-1$ 位数字串，其中有 a_{n-1} 个是有效的，所以通过在无效的 $n-1$ 位数字串后面加上一个0就可以得到 $10^{n-1} - a_{n-1}$ 个 n 位的有效数字串。

因为所有的 n 位有效数字串都用这两种方式之一产生，所以存在

$$\begin{aligned} a_n &= 9a_{n-1} + (10^{n-1} - a_{n-1}) \\ &= 8a_{n-1} + 10^{n-1} \end{aligned}$$

个 n 位有效数字串。

例5中的递推关系在许多不同的场合都可以见到。

例5 求关于 C_n 的递推关系，其中 C_n 是通过将 $n+1$ 个数 $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$ 的乘积中加括号来规定乘法的次序的方式数。例如， $C_3 = 5$ ，因为对 $x_0 \cdot x_1 \cdot x_2 \cdot x_3$ 有5种加括号的方式来规定乘法的次序：

$$\begin{aligned} &((x_0 \cdot x_1) \cdot x_2) \cdot x_3 \quad (x_0 \cdot (x_1 \cdot x_2)) \cdot x_3 \quad (x_0 \cdot x_1) \cdot (x_2 \cdot x_3) \\ &x_0 \cdot ((x_1 \cdot x_2) \cdot x_3) \quad x_0 \cdot (x_1 \cdot (x_2 \cdot x_3)) \end{aligned}$$

解 为了求关于 C_n 的递推关系，我们注意到无论怎样在 $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$ 中插入括号，总有一个“ $\cdot$ ”运算符留在所有括号的外边，即执行最后一次乘法的运算符。[例如，在 $(x_0 \cdot (x_1 \cdot x_2)) \cdot x_3$ 中的最后一个运算符，在 $(x_0 \cdot x_1) \cdot (x_2 \cdot x_3)$ 中的第二个运算符。]这个最后的运算符出现在 $n+1$ 个数中的两个数之间，比如说 x_k 和 x_{k+1} 之间。当最后的运算符出现在 x_k 和 x_{k+1} 之间时，存在 $C_k C_{n-k-1}$ 种方式插入括号来确定 $n+1$ 个数被乘的次序，因为 C_k 种方式在乘积 $x_0 \cdot x_1 \cdot \cdots \cdot x_k$ 中插入括号来确定这 $k+1$ 个数的乘法次序，所以有 C_{n-k-1} 种方式在乘积 $x_{k+1} \cdot x_{k+2} \cdot \cdots \cdot x_n$ 中插入括号来确定这 $n-k$ 个数的乘法次序。由于这个最后的运算符可能出现在 $n+1$ 个数的任两个数之间，所以

$$\begin{aligned} C_n &= C_0 C_{n-1} + C_1 C_{n-2} + \cdots + C_{n-2} C_1 + C_{n-1} C_0 \\ &= \sum_{k=0}^{n-1} C_k C_{n-k-1} \end{aligned}$$

注意初始条件是 $C_0 = 1$ 和 $C_1 = 1$ 。

这个递推关系可以用生成函数的方法求解，这种方法将在8.4节讨论。可以证明 $C_n = C(2n, n)/(n+1)$ (见8.4节的练习43) 并且 $C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$ (参见[GrKnPa94])。序列 $\{C_n\}$ 是卡特兰

(Catalan)数的序列。这个序列以尤金·查尔斯·卡塔兰命名，它是除了上例之外的许多不同计数问题的解(细节见[MiRo91]或[Ro84a]中有关卡塔兰数的一章)。

8.1.3 算法与递推关系

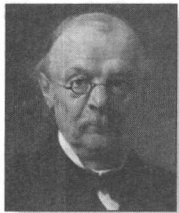
递推关系在一些算法研究和算法复杂度方面起着重要作用。在8.3节，我们将说明如何使用递推关系分析分而治之算法的时间复杂度，如在5.4节中的归并排序算法。可以在8.3节看到，分治算法递归地将一个问题分解为一个固定数量的非重叠的子问题，直到子问题简单到可以直接求解。我们在本节的最后将介绍另一种算法范式——**动态规划**，它可以有效地用于解决许多优化问题。

遵循动态规划范式的算法是将问题递归地分解为更简单的重叠子问题，通过子问题的解决来解决原问题。一般地，递推关系用于通过子问题求解找到全局问题的解决方法。动态规划已经用于解决广泛领域的一些重要问题，包括经济、计算机视觉、语音识别、人工智能、计算图形学和生物信息学。在本节中我们将说明运用动态规划设计算法解决调度问题。在此之前，我们将介绍动态规划这个名称的有趣来源，它由数学家理查德·贝尔曼在20世纪50年代提出。贝尔曼当时在RAND公司工作，参与美国军方的一个项目。当时，美国国防部对数学研究有反感。为了保证获得资助，贝尔曼必须想一个与数学无关的用于解决调度和规划问题的名字。他决定用一个形容词——**动态**。因为“使用动态这个单词不可能有贬义”，他认为“动态规划是连国会都不会反对的”。

动态规划实例 说明动态规划的问题与3.1节中例7相关。那个问题的目标是在一个讲座厅中安排尽可能多的讲座。这些讲座预设了开始和结束时间。一旦一个讲座开始，它将进行直到结束。两个讲座不可以安排在同一时间段内，一个讲座可从另一个讲座结束时开始。在5.1节例12我们提出了一种贪心算法可以得到一个优化的安排。现在我们的目标不是安排尽可能多的讲座，而是尽可能多地合并已规划讲座的参与者。

我们形式化这个问题：设有 n 个讲座，讲座 j 开始于时间 t_j ，结束于时间 e_j ，有 w_j 个学生参与。我们需要规划最大的参与学生人数。即我们希望规划一组讲座使得在所有安排的讲座中 w_j 之和最大。(注意，当一个学生参与了多个讲座时，这个学生通过他参与的讲座数来计数。)我们用 $T(j)$ 来表示由一个优化调度得到的前 j 场讲座的最大参与总数， $T(n)$ 就是一个优化调

Links



©Paul Fearn/Alamy Stock Photo

尤金·查尔斯·卡塔兰(Eugène Charles Catalan, 1814—1894) 卡塔兰出生在布鲁日，当时是法国的一部分。他的父亲在巴黎成为一个成功的建筑师，当时尤金还是一个小男孩。由于希望跟随父亲的脚步，卡塔兰进入巴黎学校学习设计。15岁时，他赢得了一个给他在设计学校的同学教几何的工作。毕业后，卡塔兰进入了一所美术学院，但因为他对数学的兴趣，他的导师建议他进入巴黎理工学院。因此，他成为理工学院的一名学生，但学习一年后，他因为政治原因而被开除，然而，他又重新入学，并于1835年毕业，还赢得了马恩河畔沙隆学院的一个职位。

1838年，卡塔兰回到巴黎，在那里，他与另外两位数学家斯特姆和刘维尔成立了一个预科学校。从事教学工作很短的时间后，他在巴黎综合理工学院获得一个职位。1841年，他获得了理工学院博士学位，但他赞成法国共和国的政治活动影响了他的职业前景。1846年卡塔兰在查理曼学院得到了一个职位，1849年他被委派到圣路易斯中学。然而，卡塔兰没有按要求对新皇帝路易-拿破仑·波拿巴效忠宣誓，并因此失去了工作。在这之后的13年中，他一直没有固定的工作。1865年，他被任命为比利时列日大学的数学主任，他在这个位置直到1884年退休。

卡塔兰对数论和连分数的相关课题做出了许多贡献。他在解决使用非交叉对角线分割多边形成为三角形问题时，定义了卡塔兰数。他的卡塔兰猜想也非常著名。这个猜想提出只有8和9是唯一的连续整数幂，这直到2003年才被证明。卡塔兰写了许多教科书，其中有的颇为流行并且有的出版了多达12个版本。也许这本教科书将有出版第12版的一天！

度得到的对于所有 n 个讲座的最大参与总数。

我们首先将讲座结束时间升序排序。此后，我们重新编号讲座， $e_1 \leq e_2 \leq \dots \leq e_n$ 。我们说两个讲座是相容的当它们能成为一个规划的一个部分。即，它们的时间不会重叠。（不同于一个结束而同时另一个开始。）对于 $e_i \leq s_j$ ，我们定义 $p(j)$ 为最大整数 i ， $i < j$ ，如果这个整数存在；否则 $p(j) = 0$ 。这样，如果存在，讲座 $p(j)$ 是与讲座 j 相容的在讲座 j 前结束的结束最晚的讲座。否则 $p(j) = 0$ ，这样的讲座不存在。

例 6 考虑 7 个讲座开始和结束时间，如图 5 所示。

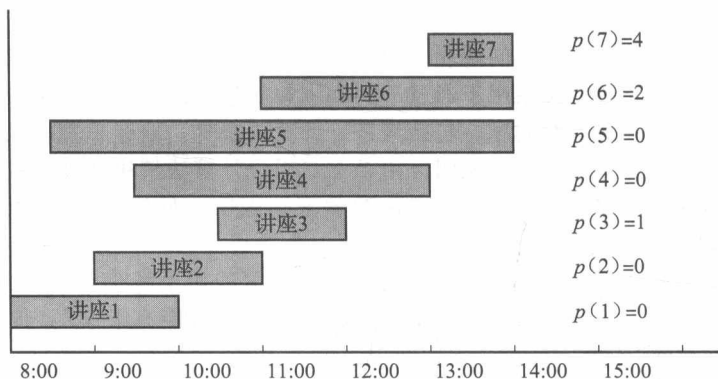


图 5 一个展示 $p(h)$ 值的讲座规划

讲座 1：开始 8:00，结束 10:00

讲座 3：开始 10:30，结束 12:00

讲座 2：开始 9:00，结束 11:00

讲座 4：开始 9:30，结束 13:00

Links



©Alfred Eisenstaedt/The LIFE Picture Collection/Getty Images

理查德·贝尔曼 (Richard Bellman, 1920—1984) 贝尔曼出生在布鲁克林，他的父亲开了一家杂货店。还是一个孩子时，他花了很多时间在纽约的博物馆和图书馆。高中毕业后，他在布鲁克林学院学习数学，并于 1941 年毕业。他开始在约翰霍普金斯大学读研究生，但由于战争，他离开约翰霍普金斯大学，转而在威斯康星大学教电子学。他在威斯康星大学继续数学研究，并于 1943 年获得了硕士学位。后来，贝尔曼进入普林斯顿大学，针对一个特殊的美国陆军计划进行教学。1944 年年底，他应征入伍。他被分配到洛斯阿拉莫斯国家实验室参与曼哈顿计划。在那里，他研究理论物理。战争结束后，他回到普林斯顿大学，并在 1946 年获得博士学位。

在普林斯顿大学任教较短的时间后，他转到美国斯坦福大学，并获得了终身教职。在斯坦福大学，他着迷于数论研究。然而，贝尔曼更集中于现实世界中所产生的数学问题。1952 年，他加入了兰德公司，研究多级决策过程、运筹学问题及其在社会科学和医学中的应用。他在兰德公司参与了多个军事项目的研究。1965 年，他离开兰德公司，成为南加州大学数学、电气、生物医学工程和医学教授。

在 20 世纪 50 年代，贝尔曼率先在大范围的场景下使用早先发明的动态规划。他在随机控制流程方面的工作也是众所周知的，他引入了贝尔曼方程。他创造了维数灾难一词，用于描述在空间中增加额外维度引起的指数数量增加的问题。他写了惊人数量的书籍和合著论文，其中大多数针对工业生产和经济系统。他的工作使得计算技术在各种领域得到应用，这些领域从空间飞行器的制导系统设计，到网络优化，甚至涉及病虫害防治。

不幸的是，1973 年贝尔曼被诊断出患有脑瘤。虽然被成功摘除，但并发症给他留下了严重残疾。幸运的是，在余下的 10 年里，他尝试着继续研究和写作。贝尔曼获得了许多奖金和奖项，其中包括第一届诺伯特·维纳应用数学奖和 IEEE 金奖荣誉。他被选入美国国家科学院。他被认为是具有很高的成就、勇气和令人钦佩素质。贝尔曼是两个孩子的父亲。

讲座 5: 开始 8:30, 结束 14:00 讲座 7: 开始 13:00, 结束 14:00

讲座 6: 开始 11:00, 结束 14:00

对于 $j=1, 2, \dots, 7$, 计算 $p(j)$ 。

解 我们可以得到 $p(1)=0$ 和 $p(2)=0$ 。因为没有讲座结束时间在这两个讲座开始之前。可得到 $p(3)=1$, 因为讲座 3 和讲座 1 是相容的。但讲座 3 和讲座 2 不相容。 $p(4)=0$, 因为讲座 4 与讲座 1、2、3 都不相容。 $p(5)=0$, 因为讲座 5 与讲座 1、2、3、4 都不相容。 $p(6)=2$, 因为讲座 6 和讲座 2 是相容的。但讲座 6 和讲座 3、4、5 不相容。最后 $p(7)=4$, 因为讲座 7 和讲座 4 是相容的。但讲座 7 和讲座 5、6 不相容。 ◀

为了设计一个解决这个问题的动态规划算法, 我们首先提出一个关键的递推关系。首先注意 $j \leq n$ 。对于前 j 个讲座, 有两种可能的优化调度(注意, 我们已经将 n 个讲座按结束时间升序排序): (i) 讲座 j 属于优化调度; (ii) 它不属于。

情况(i): 我们知道讲座 $p(j)+1, \dots, j-1$ 不可能属于这个规划, 因为这些讲座与讲座 j 都不相容。进一步, 在优化调度中的其他讲座必定包括了讲座 $1, 2, \dots, p(j)$ 的一个优化调度。因为如果对于 $1, 2, \dots, p(j)$ 有更好的优化调度, 通过加上讲座 j , 我们将得一个比总体优化调度更好的规划。所以, 在情况(i)下, $T(j)=w_j+T(p(j))$ 。

情况(ii): 当讲座 j 不属于一个优化调度时, 讲座 $1, 2, \dots, j$ 的优化调度就与讲座 $1, 2, \dots, j-1$ 的一样。因此, 在情况(ii)下, $T(j)=T(j-1)$ 。结合情况(i)和(ii), 可得到一个递推关系:

$$T(j) = \max(w_j + T(p(j)), T(j-1))$$

现在我们得到这个递推关系, 我们将设计一个有效算法(算法 1)来计算最大的参与总数。在计算之后, 通过保存每一个 $T(j)$ 值来保证这个算法是有效的。这样可以只计算 $T(j)$ 一次。如果不这样, 算法将有指数级最坏情况时间复杂度。保存每一次计算值的过程称为记忆, 这对于提高递归算法效率是很重要的技术。

算法 1 调度讲座的动态规划算法

Procedure Maximum Attendees($s_1, s_2, \dots, s_n$: 讲座的开始时间; $e_1, e_2, \dots, e_n$: 讲座的结束时间;

$w_1, w_2, \dots, w_n$: 讲座的参与人数)

将讲座按结束时间排序, 并重新标记讲座, 保证 $e_1 \leq e_2 \leq \dots \leq e_n$

for $j := 1$ **to** n

if 没有任务 $i(i < j)$ 与任务 j 相容;

$p(j) := 0$

else $p(j) := \max\{i \mid i < j \text{ 并且任务 } i \text{ 与任务 } j \text{ 相容}\}$

$T(0) := 0$

for $j := 1$ **to** n

$T(j) := \max(w_j + T(p(j)), T(j-1))$

return $T(n)$ ($T(n)$ 是最大的参与人数)

在算法 1 中, 我们通过一个讲座调度方案确定最大的参与人数, 但我们不能找到获得最大人数的调度方案。为了找到这个调度方案, 我们用到这个事实: 对于前 j 个讲座, 讲座 j 属于一个优化方案当且仅当 $w_j + T(p(j)) \geq T(j-1)$ 。我们将这个问题留作练习 53, 基于这个观察设计一个算法, 以确定哪些讲座应该在获得最大参与人数的调度方案中。

算法 1 是动态规划的一个好例子, 因为通过重叠的子问题的优化方案找到了最大参与人数。每一个子问题确定前 j 个讲座的最大参与人数, $1 \leq j \leq n-1$ 。其他动态规划例子参见练习 56 和 57 以及补充练习 14 和 17。